

Parcial Ii - Base De Datos Solicitudes De Visa Para Colombia 2017

Ana María Camacho Castaño

Fabian Alberto Bedoya

Maicol Smith Bueno

Jorge Iván Marín Cifuentes

Alejandro Taborda Sepúlveda

Mayo 2026

CIAF

Pereira - Risaralda

Inteligencia Artificial

Tabla de Contenido

1. Introducción.....	3
2. Importaciones Iniciales	4
3. Carga del CSV y convertirlo en DataFrame.....	4
4. Análisis #1: Top Nacionalidades con más solicitudes.....	6
5. Análisis #2: Solicitudes por año	7
6. Análisis #3: Distribución de solicitudes por Sexo	8
7. Análisis #4: Edad promedio de los solicitantes.....	9
8. Menú Interactivo.....	10

1. Introducción

En la actualidad, el análisis de datos se ha convertido en una herramienta fundamental para la toma de decisiones en múltiples áreas como la economía, la tecnología y la gestión pública. En este contexto, el uso de estructuras como los DataFrames permite organizar, procesar y analizar grandes volúmenes de información de manera eficiente.

Un DataFrame es una estructura de datos bidimensional, similar a una hoja de cálculo, que forma parte de la librería Pandas en el lenguaje de programación Python. Esta herramienta facilita la manipulación de datos mediante operaciones como filtrado, agrupación, limpieza y transformación, permitiendo convertir datos crudos en información útil.

El análisis de bases de datos consiste en examinar, organizar e interpretar conjuntos de datos con el fin de identificar patrones, tendencias y relaciones relevantes. Este proceso incluye etapas como la carga de datos, la limpieza de valores inconsistentes, la exploración inicial y la aplicación de técnicas estadísticas o visuales para obtener conclusiones significativas.

En el caso del análisis de solicitudes de visa en Colombia, el uso de DataFrames permite estructurar variables como año, nacionalidad, sexo y vocación de permanencia, facilitando el estudio del comportamiento migratorio. A través de este enfoque, es posible generar conocimiento valioso que contribuya a la comprensión de fenómenos sociales y a la toma de decisiones basadas en datos.

2. Importaciones Iniciales

```
import pandas as pd
import matplotlib.pyplot as plt
from datetime import datetime
```

Cada importación tiene una función específica en nuestro código:

pandas	→ motor de análisis de datos (tipo Excel pero vitaminado).
matplotlib.pyplot	→ capa de visualización (gráficas).
datetime	→ manejo de fechas (clave para calcular edades).

3. Carga del CSV y convertirlo en DataFrame

Lee un archivo Excel y lo convierte en un DataFrame (tabla estructurada), para manipular la información y generar correctamente nuestros análisis, se deja una función que englobe toda la transformación del Data Frame incluyendo las columnas.

```
def cargar_datos(ruta):
    df = pd.read_excel(ruta)
```

- Estandariza nombres → más limpio y fácil de trabajar
- Evita errores tipo "Fecha Nacimiento" vs "fecha_nacimiento"

```
df.columns = [
    "anio",
    "nacionalidad",
    "sexo",
    "fecha_nacimiento",
    "vocacion",
    "numero"
]
```

- Convierte texto → formato fecha
- errors='coerce' = si algo falla → lo convierte en NaT (valor nulo)

```
df["fecha_nacimiento"] = pd.to_datetime(df["fecha_nacimiento"], errors='coerce')
```

**Errors='coerce' = Forzar la conversión de la columna a tipo fecha, y si falla lo convertir a nulo.*

de esta manera dejamos toda la transformación de la data dentro de una Función llamada “**cargar_datos**” y como parámetro enviamos **ruta** corresponde donde se encuentra nuestro CSV para convertirlo.

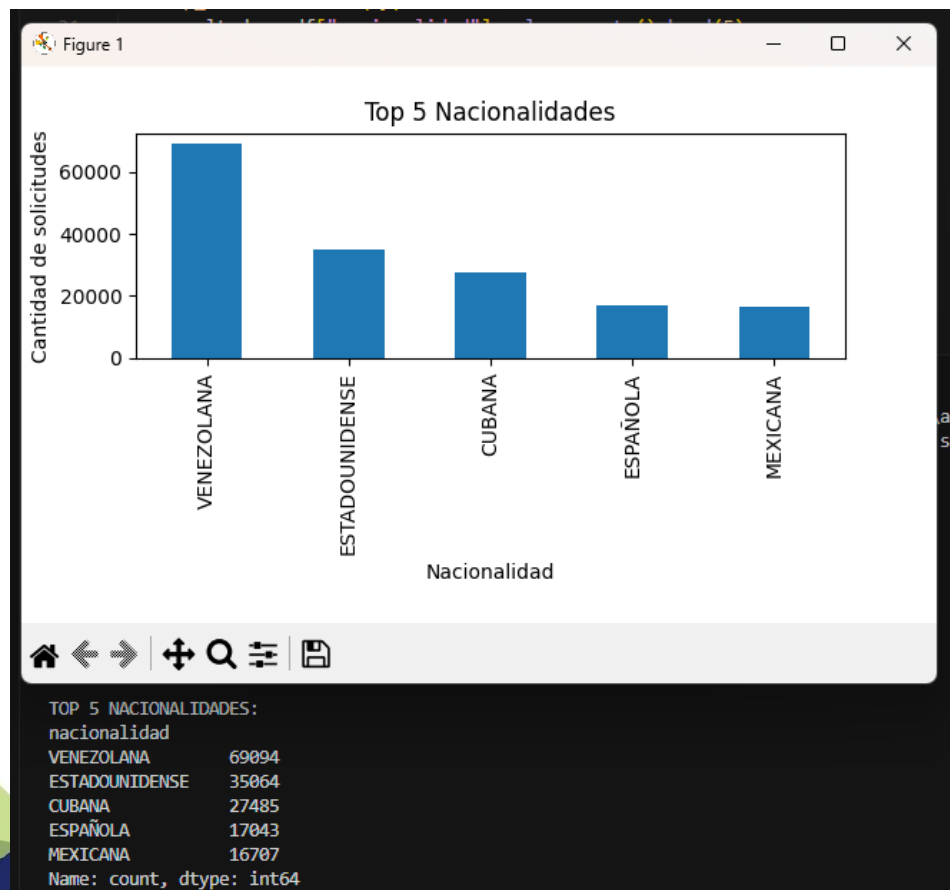
```
# =====  
# CARGA Y LIMPIEZA DE DATOS  
# =====  
def cargar_datos(ruta):  
    df = pd.read_excel(ruta)  
  
    # Renombrar columnas (más limpias)  
    df.columns = [  
        "anio",  
        "nacionalidad",  
        "sexo",  
        "fecha_nacimiento",  
        "vocacion",  
        "numero"  
    ]  
  
    # Convertir fechas  
    df["fecha_nacimiento"] = pd.to_datetime(df["fecha_nacimiento"], errors='coerce')  
  
    return df
```

4. Análisis #1: Top Nacionalidades con más solicitudes.

Todo inicia con la variable **Resultado**, esta realiza conteo y un ranking de las primeras nacionalidades con más solicitudes de visa, se realiza en una sola función tanto el cálculo como la gráfica automática con Matplotlib, para facilidad de interpretación y buenas prácticas.

```
# =====  
# ANÁLISIS 1: TOP NACIONALIDADES  
# =====  
def top_nacionalidades(df):  
    resultado = df["nacionalidad"].value_counts().head(5)  
  
    print("\nTOP 5 NACIONALIDADES:")  
    print(resultado)  
  
    resultado.plot(kind='bar')  
    plt.title("Top 5 Nacionalidades")  
    plt.xlabel("Nacionalidad")  
    plt.ylabel("Cantidad de solicitudes")  
    plt.show()
```

RESULTADO

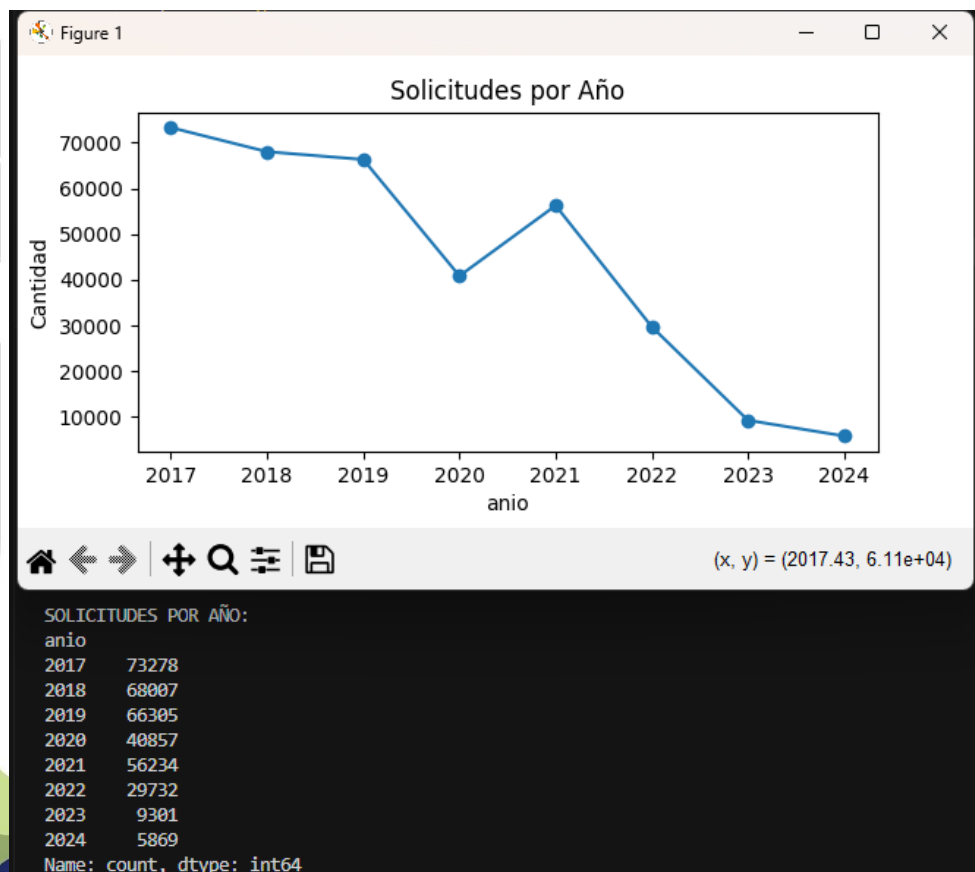


5. Análisis #2: Solicitudes por año

Cuenta la cantidad de solicitudes por año y las organiza de manera cronológica por medio del método `.sort_index()` para luego generar una grafica de líneas con su resultado **Kind='line'** y el marcador de cada punto de la grafica por medio de **marker='o'**

```
# =====  
# ANÁLISIS 2: SOLICITUDES POR AÑO  
# =====  
def solicitudes_por_año(df):  
    resultado = df["anio"].value_counts().sort_index()  
  
    print("\nSOLICITUDES POR AÑO:")  
    print(resultado)  
  
    resultado.plot(kind='line', marker='o')  
    plt.title("Solicitudes por Año")  
    plt.ylabel("Cantidad")  
    plt.show()
```

RESULTADO

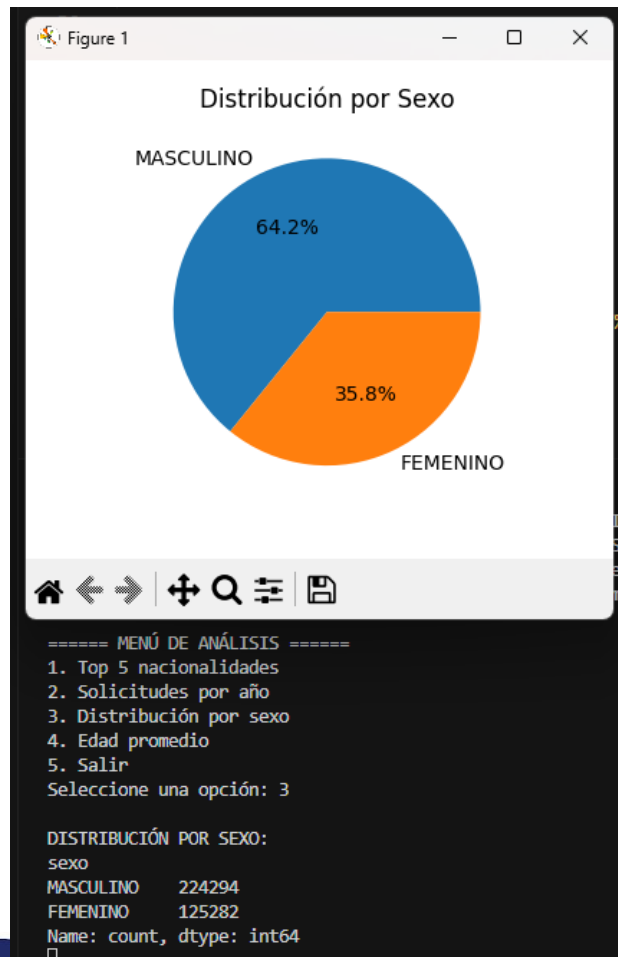


6. Análisis #3: Distribución de solicitudes por Sexo

Realiza la distribución del total de solicitudes por sexo del solicitante, para posteriormente graficarla de acuerdo con el % de participación tipo 'Pie'

```
# =====  
# ANÁLISIS 3: DISTRIBUCIÓN POR SEXO  
# =====  
def distribucion_sexo(df):  
    resultado = df["sexo"].value_counts()  
  
    print("\nDISTRIBUCIÓN POR SEXO:")  
    print(resultado)  
  
    resultado.plot(kind='pie', autopct='%1.1f%%')  
    plt.title("Distribución por Sexo")  
    plt.ylabel("")  
    plt.show()
```

RESULTADO

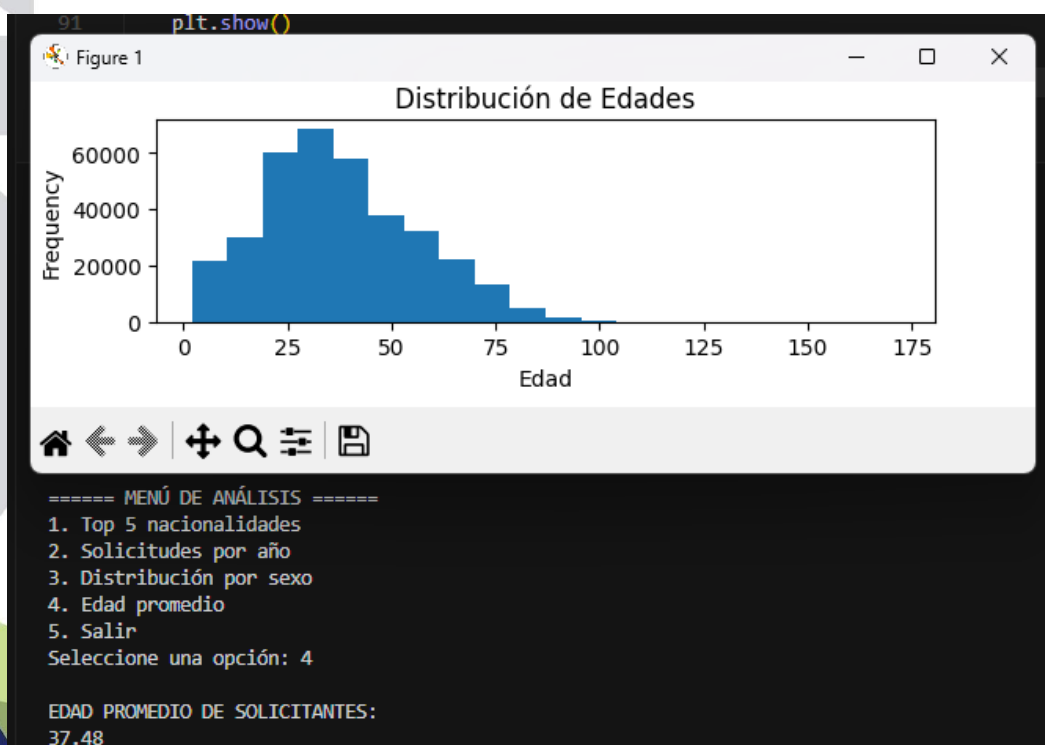


7. Análisis #4: Edad promedio de los solicitantes.

Esta medida crea la edad promedio de mis solicitantes de acuerdo con su fecha de nacimiento con diferencia a la fecha actual **hoy = datetime.now()**, de esta manera se genera una diferencia correspondiente a los años y luego se genera una gráfica tipo histograma.

```
# =====  
# ANÁLISIS 4: EDAD PROMEDIO  
# =====  
def edad_promedio(df):  
    hoy = datetime.now()  
  
    df["edad"] = df["fecha_nacimiento"].apply(  
        lambda x: hoy.year - x.year if pd.notnull(x) else None  
    )  
  
    promedio = df["edad"].mean()  
  
    print("\nEDAD PROMEDIO DE SOLICITANTES:")  
    print(round(promedio, 2))  
  
    df["edad"].plot(kind='hist', bins=20)  
    plt.title("Distribución de Edades")  
    plt.xlabel("Edad")  
    plt.show()
```

RESULTADO



8. Menú Interactivo

Se crea una función que llamamos en el MAIN para que se ejecute el menuy este a su vez, ejecuta cada una de las funciones con los análisis creados posteriormente, este menú siempre permanece activo gracias a un loop infinito hasta que le oprimas en la opción “Salir”

```
# =====  
# MENÚ INTERACTIVO  
# =====  
def menu(df):  
    while True:  
        print("\n===== MENÚ DE ANÁLISIS =====")  
        print("1. Top 5 nacionalidades")  
        print("2. Solicitudes por año")  
        print("3. Distribución por sexo")  
        print("4. Edad promedio")  
        print("5. Salir")  
  
        opcion = input("Seleccione una opción: ")  
  
        if opcion == "1":  
            top_nacionalidades(df)  
        elif opcion == "2":  
            solicitudes_por_anio(df)  
        elif opcion == "3":  
            distribucion_sexo(df)  
        elif opcion == "4":  
            edad_promedio(df)  
        elif opcion == "5":  
            print("Saliendo...")  
            break  
        else:  
            print("Opción inválida")
```

RESULTADO

```
===== MENÚ DE ANÁLISIS =====  
1. Top 5 nacionalidades  
2. Solicitudes por año  
3. Distribución por sexo  
4. Edad promedio  
5. Salir  
Seleccione una opción: |
```